PART 1

This project aims to simulate and implement five algorithms: First Come First Served (FCFS), Shortest Job First (SJF), Priority Scheduling (NonPreemptive, Preemptive), and Round Robin (RR). It also measures the performance of these algorithms by calculating (Waiting Time) and (Turnaround Time).

I have divided the project into three files to facilitate programming, maintenance, .and access to the desired goal in a simpler way

The first file, main

contains lines for printing and receiving user input (such as Arrival Time, Burst Time), in addition to calling the algorithms in the third file. It also includes (Part 2) of the project, which is a simulation of the Banker's Algorithm

The algorithm requires the input of the number of operations and resources, the maximum demand for each operation, the current allocation, and the currently available resources. It also performs a security check and displays the result to .the user

The second file, Process.h

I defined a struct containing user inputs such as (processID, arrivalTime, burstTime, ProcPriority)

to calculate (CompletionTime, WaitingTime, TurnaroundTime) for us, and I declaration the algorithms in addition to the standard print function printSchedulerResults to print after each algorithm

The third file, Scheduler.cpp

actually contains all the algorithm functions FCFS(), SJF(), Preemptive(), NonPreemptive(), and RR()

Each code within these functions is responsible for applying its own scheduling logic and calculating the final performance metrics for each process .(CompletionTime, WaitingTime, TurnaroundTime)

print_gantt_chart(): Responsible for building and displaying a Gantt chart that .represents the chronological sequence of process execution

printSchedulerResults(): Responsible for displaying the final results of the algorithm in a standard table format, including calculating and displaying .performance averages (Average WT, Average TAT)

1) FCFS algorithms ”First come, first served“

The inputs are a list of operations with arrival time and burst time.

Outputs:

• Waiting time for each operation.

• Turnaround time for each operation.

• Average waiting and turnaround time.

• Gantt chart (order of operations by arrival time).

How it works:

Processes wait in a queue in order of their arrival time (AT).

When the CPU becomes free, the process with the lowest AT is selected to run.

```
=== Part 1: CPU Scheduling Project ===

Please Enter number of processes: 5

 Please Enter process details:

Process 1:
  Arrival Time: 4
  Burst Time: 8
  Priority (lower number = higher priority): 4

Process 2:
  Arrival Time: 0
  Burst Time: 5
  Priority (lower number = higher priority): 2

Process 3:
  Arrival Time: 6
  Burst Time: 6
  Priority (lower number = higher priority): 3

Process 4:
  Arrival Time: 2
  Burst Time: 3
  Priority (lower number = higher priority): 1

Process 5:
  Arrival Time: 8
  Burst Time: 2
  Priority (lower number = higher priority): 2


 Please Enter time quantum for Round Robin(RR) : 2
```

```
--- First-Come, First-Served Scheduling ---
Process | AT    | BT    | Priority| CT   | TAT   | WT
-----------------------------------------------------------
P2      | 0     | 5     | 2       | 5    | 5     | 0
P4      | 2     | 3     | 1       | 8    | 6     | 3
P1      | 4     | 8     | 4       | 16   | 12    | 4
P3      | 6     | 6     | 3       | 22   | 16    | 10
P5      | 8     | 2     | 2       | 24   | 16    | 14
-----------------------------------------------------------
Average Waiting Time: 6.20
Average Turnaround Time: 11.00

--- Gantt Chart ---
|--P2--|--P4--|--P1--|--P3--|--P5--|
0 5 8 16 22 24
```

---

2) SJF algorithms " Shortest Job Frist "

The inputs are a list of operations with arrival time and burst time.

Outputs:

• Calculation of waiting and rotation times for each operation

• .Average times

• .Gantt chart showing the order of operations by execution time

How it works:

The process with the shortest execution time among the processes currently in memory is selected, until it finishes and the next process begins.

```
--- Shortest Job First (Non-Preemptive) Scheduling ---
Process | AT     | BT     | Priority| CT   | TAT    | WT
-----------------------------------------------------------
P1      | 4      | 8      | 4       | 24   | 20     | 12
P2      | 0      | 5      | 2       | 5    | 5      | 0
P3      | 6      | 6      | 3       | 16   | 10     | 4
P4      | 2      | 3      | 1       | 8    | 6      | 3
P5      | 8      | 2      | 2       | 10   | 2      | 0
-----------------------------------------------------------
Average Waiting Time: 3.80
Average Turnaround Time: 8.60

--- Gantt Chart ---
|--P2--|--P4--|--P5--|--P3--|--P1--|
0 5 8 10 16 24
```

---

3) Priority Scheduling (Preemptive)

Inputs are a list of operations with arrival time, priority, and execution time.

Outputs:

• Wait times and turnaround times for each process.

• Averages.

• Gantt chart showing how interruptions occur.

How it works:

If a new process with a higher priority arrives while another process is being executed, the current process is stopped and the higher priority process is started immediately.

```
--- Priority (Preemptive) Scheduling ---
Process | AT     | BT     | Priority| CT   | TAT    | WT
-----------------------------------------------------------
P1      | 4      | 8      | 4       | 24   | 20     | 12
P2      | 0      | 5      | 2       | 8    | 8      | 3
P3      | 6      | 6      | 3       | 16   | 10     | 4
P4      | 2      | 3      | 1       | 5    | 3      | 0
P5      | 8      | 2      | 2       | 10   | 2      | 0
-----------------------------------------------------------
Average Waiting Time: 3.80
Average Turnaround Time: 8.60

--- Gantt Chart ---
|--P2--|--P4--|--P2--|--P5--|--P3--|--P1--|
0 2 5 8 10 16 24
```

---

4) Priority Scheduling (Non-Preemptive)

The inputs are a list of operations with arrival time and priority (the smaller the number, the higher the priority).

Outputs:

• Waiting and turnaround time for each operation.

• Averages.

• Gantt chart showing the order of execution according to the priority of operations.

How it works:

The highest priority task is selected and executed completely before moving on to the next task, without interruption.

```
--- Priority (Non-Preemptive) Scheduling ---
Process | AT     | BT     | Priority| CT   | TAT    | WT
-------------------------------------------------------------
P1      | 4      | 8      | 4       | 24   | 20     | 12
P2      | 0      | 5      | 2       | 5    | 5      | 0
P3      | 6      | 6      | 3       | 16   | 10     | 4
P4      | 2      | 3      | 1       | 8    | 6      | 3
P5      | 8      | 2      | 2       | 10   | 2      | 0
-------------------------------------------------------------
Average Waiting Time: 3.80
Average Turnaround Time: 8.60

--- Gantt Chart ---
|--P2--|--P4--|--P5--|--P3--|--P1--|
0 5 8 10 16 24
```

---

5) Round Robin (RR)

The inputs are a list of operations with execution time and arrival time, as well as the allocation time (TIME_QUANTUM) specified at the top.

Outputs:

• Waiting and turnaround times for each operation.

• Averages.

• A Gantt chart showing the rotation between operations in each time period.

How it works:

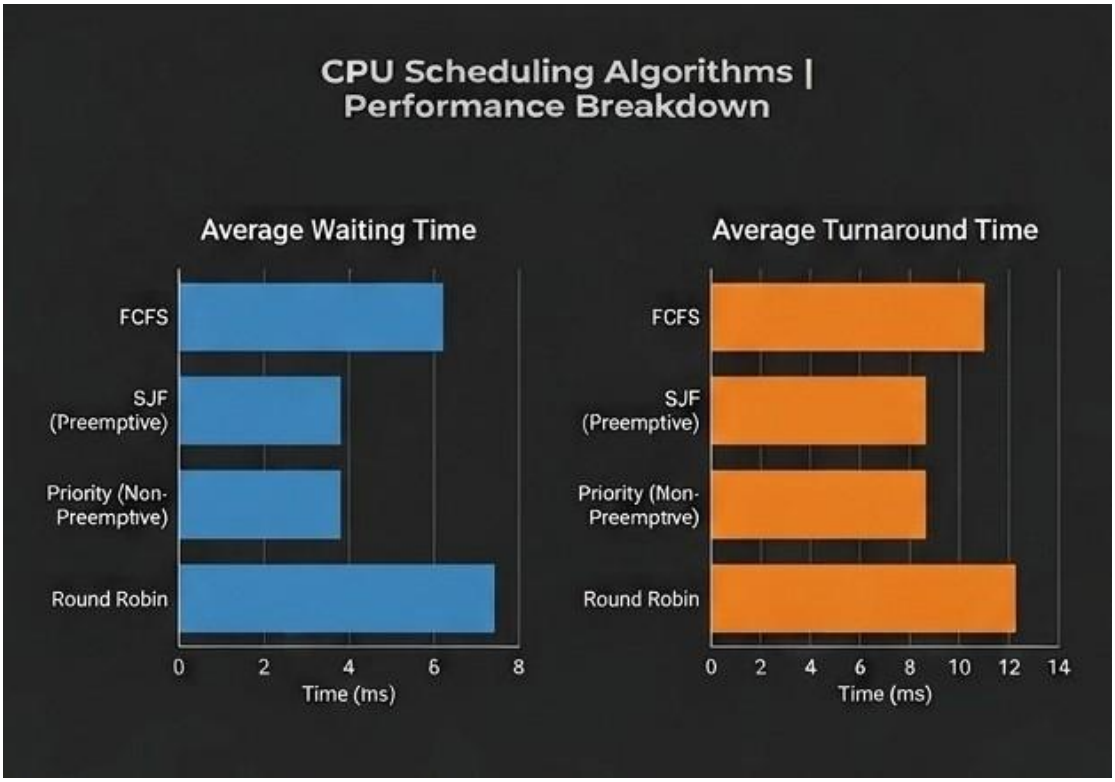
Each process is allocated a specific amount of time (quantum) for execution, then paused and given the opportunity to the next process, and so on in rotation until all processes are completed.

```
--- Round Robin Scheduling ---
Process | AT     | BT     | Priority| CT   | TAT    | WT
-------------------------------------------------------------
P1      | 4      | 8      | 4       | 24   | 20     | 12
P2      | 0      | 5      | 2       | 12   | 12     | 7
P3      | 6      | 6      | 3       | 22   | 16     | 10
P4      | 2      | 3      | 1       | 9    | 7      | 4
P5      | 8      | 2      | 2       | 14   | 6      | 4
-------------------------------------------------------------
Average Waiting Time: 7.40
Average Turnaround Time: 12.20

--- Gantt Chart ---
|--P2--|--P4--|--P2--|--P1--|--P4--|--P3--|--P2--|--P5--|--P1--|--P3--|--P1--|--P3--|--P1--|
0 2 4 6 8 9 11 12 14 16 18 20 22 24
```

the link for code https://onlinegdb.com/CLX27AUxk

Algorithm	Avg Waiting Time	Avg Turnaround Time	Basic Idea	Observation
FCFS	6.20	11.00	the process is run on a first come	Simple but unfair for short processes because .the wait is long
SJF	3.80	8.60	Shortest job runs first.	Offers the best overall performance and the best average times, but requires prior knowledge burst time
Priority Scheduling (Preemptive)	3.80	8.60	Highest priority runs first.	It handles important tasks first, so low-priority tasks may be delayed. Therefore, the result is the same as SJF.
Priority Scheduling (Non-Preemptive)	3.80	8.60	Highest priority runs first **without interruption**.	Faster response for urgent jobs and Similar times but more .switching
Round Robin (RR)	7.40	12.20	Each process gets equal time slices.	Fair and responsive, and best for multitasking, its drawback is long wait times due to numerous switches, which affect its efficiency.



PART 2

```
=== Part 2: Banker's Algorithm ===
Please Enter the number of resource types: 3

Enter Maximum Demand Matrix (5x3):
  P0 Max Need: 7 5 3
  P1 Max Need: 3 2 2
  P2 Max Need: 9 0 2
  P3 Max Need: 2 2 2
  P4 Max Need: 4 3 3

Enter Current Allocation Matrix (5x3):
  P0 Assigned: 0 1 0
  P1 Assigned: 2 0 0
  P2 Assigned: 3 0 2
  P3 Assigned: 2 1 1
  P4 Assigned: 0 0 2

Enter Available Resource Vector: 3 3 2

System is in a **SAFE STATE**.
Safe Execution Order: P1 -> P3 -> P0 -> P2 -> P4
```

Inputs:

• Number of processes: 5

• Number of resources: 3

• Allocation matrix: Represents the resources currently allocated to each process.

• Maximum matrix: Represents the maximum amount of resources that each process can request.

What the program does:

1. Calculates the total resources allocated to all processes.

2. Calculates the remaining available resources in the system after allocation.

3. Begins checking processes one by one to see if they can be executed without exceeding the available resources.

4. If the process can be executed, the following occurs:

o It prints it on the screen as being in progress.

o It releases its resources after completion (adding them to the available resources)

o It adds it to the Safe Sequence.

5. It repeats the steps until all processes are finished or it detects an unsafe condition.

Outputs:

- System status (secure or insecure).
- Security sequence of operations if the system is secure.
- Resources available after each operation is performed.

The program simulates how an operating system manages resources

If all processes are able to execute their tasks without resource conflicts, the system is shown to be in a safe state and displays the safe order of execution,

If some processes cannot be executed due to a lack of resources, the system is shown to be in an unsafe state.